

What the Sourcing Agent hands off — and exactly when it calls an LLM

Same contract-first pattern as the Investigator: a narrow toolset, a deterministic guardrail, and one clearly marked point where a model is actually invoked.

✓ Built & tested — `submission/agents/sourcing.py`

00

Where the LLM call happens

One logical call site, inside `run_sourcing()` — everything before it is plain Python, everything after it re-checks what the model did.



`compiled_agent.invoke(...)` — one or two times, never zero unless the precondition already failed

1. **Before any LLM call:** `run_sourcing()` checks its one precondition — a `risk: StockRisk` from the Investigator Agent. Missing it → `BLOCKED` immediately, no model call at all.
2. **LLM call #1** happens at `invoke_with_repair()` 's first attempt: the model reads the prompt, decides to call `list_vendor_offers` → `get_vendor_performance` → `build_vendor_options` → `recommend_vendor_option` in turn, then drafts a structured `SourcingOutput` (its `status` guess and a `reason`).
3. **LLM call #2 (conditional):** only if that structured output is missing or fails schema validation — the same error is appended to the conversation and the model is asked once more. Two failures in a row → fail closed to `BLOCKED` , no third attempt.
4. **After the LLM call:** `derive_sourcing_result()` re-reads the raw tool results captured off the message trace and recomputes `ELIGIBLE` / `BLOCKED` itself — the model's own status guess from step 2 is discarded, only its `reason` text is kept.

01

The full call chain

Every function `run_sourcing()` actually calls, in execution order. Four are the provided read-only tools; the rest are this agent's own code. Only one step is a model call.

YOUR CODE

1

run_sourcing() `sourcing.py:158`

Entry point. Checks the one precondition — `risk is None` → BLOCKED, nothing below this line runs.

2

build_sourcing_agent() `sourcing.py:88`

Compiles LangGraph's `create_react_agent(...)` bound to the 4 tools below. Skipped if an `agent=` is passed in (e.g. from a test).

3

`LLM call` **invoke_with_repair()** `_common.py`

Calls `compiled_agent.invoke(...)` — the model itself decides to make the four tool calls below, then drafts a structured answer. Retries once, with the validation error appended, if that answer is invalid; fails closed to BLOCKED after two failures.

PROVIDED TOOLS — CALLED BY THE MODEL, INSIDE STEP 3

4

list_vendor_offers(sku) `tools/vendors.py:28`

Active, non-expired offers for the SKU, sorted by price.

5

get_vendor_performance(vendor_ids) `tools/vendors.py:106`

Reliability score per vendor; eligible if ≥ 0.90 .

6

build_vendor_options(stock_risk, offers, performance) `tools/vendors.py:173`

Deterministic. Computes cost, deadline fit, and `eligible_options` for every offer.

7

recommend_vendor_option(case_id, sku, warehouse_id, options, strategy)`tools/workflow.py:29`

Deterministic. Picks one eligible option per `strategy` and writes the plain-English rationale.

YOUR CODE — AFTER THE MODEL RETURNS

8

extract_tool_evidence() `_common.py`

Pulls the raw JSON each tool call above actually returned out of the LLM's message trace.

9

VendorOfferList / VendorPerformanceList / VendorOptionList / VendorRecommendation .model_validate()

`pydantic`

Re-parses that raw JSON back into typed objects — nothing here is trusted as text.

10

derive_sourcing_result() `sourcing.py:98`

The guardrail. Reads the typed objects from step 9, not anything the model claimed, to decide ELIGIBLE vs BLOCKED.

11

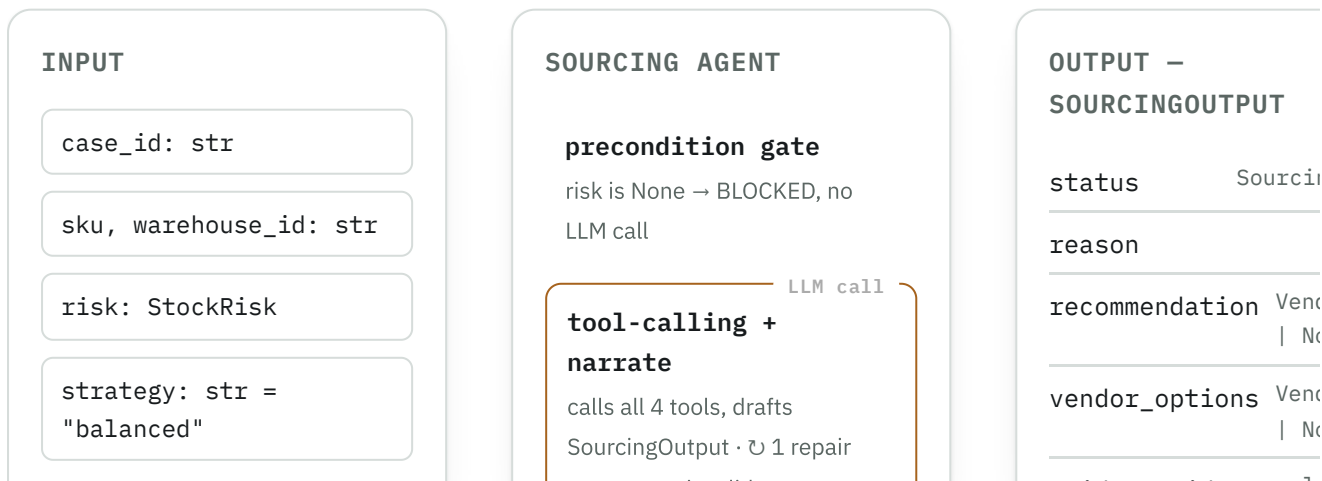
_collect_evidence_ids() `sourcing.py:136`

Gathers `evidence_id`s for the audit trail, then `run_sourcing()` returns the finished `SourcingOutput`.

02

Input → pipeline → output

The amber stage below is the only one that touches a model. Everything on either side of it is deterministic code.



attempt on invalid output

derive_sourcing_result()

recomputes ELIGIBLE /
BLOCKED from raw evidence —
LLM's status is discarded

03

Two possible shapes

Fewer exits than the Investigator, because this agent is only ever entered on an already-confirmed at-risk case.

BLOCKED

Four distinct reasons, all read from raw tool evidence: missing `risk` (no LLM call), vendor data unavailable, no option meets deadline+reliability, or no recommendation produced.

`blocked_reason`: `AGENT_FAILURE` ·
`VENDOR_OFFERS_UNAVAILABLE` ·
`VENDOR_PERFORMANCE_UNAVAILABLE` ·
`NO_ELIGIBLE_VENDOR` · `NO_RECOMMENDATION`

ELIGIBLE

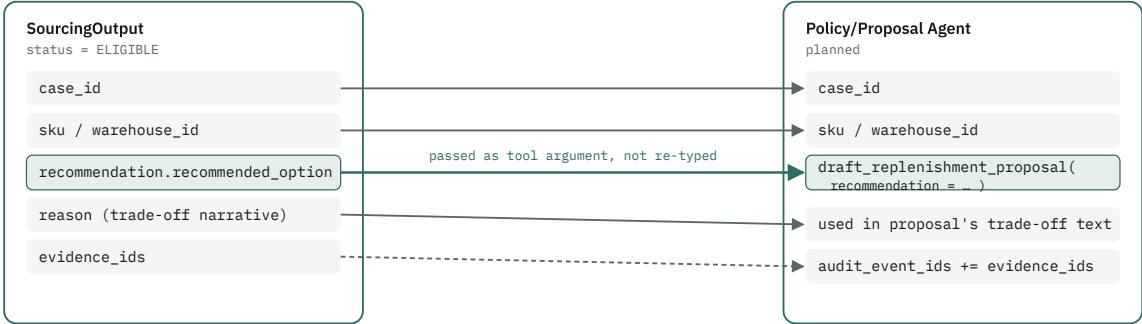
At least one vendor option clears both the deadline and the 0.90 reliability bar.
`recommendation.rationale` (LLM prose, grounded in the chosen `VendorOption`) becomes `reason`.

→ continues to Policy/Proposal Agent

04

Handoff to the Policy/Proposal Agent

Only the `ELIGIBLE` exit is wired forward. The chosen `VendorOption` — cost, vendor, arrival date — passes straight through into the proposal draft.



Claim: the recommended vendor's price, quantity, and arrival date reach the proposal exactly as `build_vendor_options` computed them — the Policy/Proposal Agent's LLM narrates policy compliance, it never re-derives cost or timing.

05

Field reference

Full shape of `SourcingOutput` (`submission/contracts.py`).

FIELD	TYPE	POPULATED WHEN
<code>case_id / sku / warehouse_id</code>	<code>str</code>	Always — echoed from
<code>strategy</code>	<code>str</code>	Always — "balanced overridden
<code>status</code>	<code>SourcingStatus</code>	Always — ELIGIBLE, BLOCKED, set by <code>derive_sourcing_</code> never by the LLM dir
<code>reason</code>	<code>str</code>	Always — LLM prose ELIGIBLE, determini message on BLOCKE